

RATE LIMITER

Работа выполнена командой:

Силевич Андрей

Бородатов Ярослав

Левина Яна

Тихомирова Дарья

Тастаков Александр

Аннотация

В данной работе рассматривается задача ограничения частоты запросов в многопользовательских распределённых системах. Реализован потокобезопасный Rate Limiter, поддерживающий несколько алгоритмов ограничения запросов: Token Bucket, Leaking Bucket и Sliding Window Log.

Основной целью исследования является сравнение указанных алгоритмов на различных типах нагрузки и определение областей их наиболее эффективного применения. В рамках работы выполнен анализ математических моделей алгоритмов, разработан нагрузочный тестер, реализованы механизмы управления временем жизни ключей (TTL), а также проведены эксперименты по измерению производительности, точности ограничения, использования памяти и устойчивости к всплескам нагрузки.

В качестве средств разработки использовались язык программирования C++23, стандартная библиотека STL, Google Test для модульного тестирования и Google Benchmark для нагрузочного тестирования.

Введение

Современные информационные системы должны обеспечивать устойчивую работу при высокой интенсивности запросов и большом количестве одновременно работающих клиентов. Для предотвращения перегрузки серверов и обеспечения справедливого распределения ресурсов широко применяются механизмы ограничения частоты запросов (Rate Limiting).

Существует несколько подходов к реализации ограничения запросов, различающихся точностью, производительностью и требованиями к ресурсам. Выбор конкретного алгоритма зависит от характера нагрузки и требований к системе.

Целью данной работы является сравнительное исследование алгоритмов Token Bucket, Leaking Bucket и Sliding Window Log в условиях различных профилей нагрузки.

Для достижения поставленной цели были поставлены следующие задачи:

- изучить существующие алгоритмы ограничения частоты запросов;
 - реализовать потокобезопасную систему Rate Limiter;
 - реализовать алгоритмы Token Bucket, Leaking Bucket и Sliding Window Log;
 - реализовать механизм TTL для удаления неактивных клиентов;
 - разработать нагрузочный тестер для моделирования различных сценариев работы;
 - провести экспериментальное сравнение алгоритмов;
 - проанализировать полученные результаты и определить особенности применения каждого подхода.
-

1) Теоретические основы ограничения частоты запросов

1.1 Понятие Rate Limiting

Rate Limiting представляет собой механизм контроля интенсивности обращений клиентов к некоторому ресурсу. Основной задачей является предотвращение перегрузки системы и обеспечение справедливого доступа к вычислительным ресурсам.

Типичный Rate Limiter принимает решение о разрешении или отклонении каждого запроса на основании ранее накопленной статистики.

Пусть λ — интенсивность входящего потока запросов, r — допустимая интенсивность обработки запросов, а t — длительность временного интервала. Тогда ожидаемое число входящих запросов за интервал t равно: $N_{in}(t) = \lambda t$.

При этом Rate Limiter должен ограничивать число разрешённых запросов величиной: $N_{allow}(t) \leq \min(\lambda t, rt)$.

Если $\lambda \leq r$, система должна пропускать большую часть запросов. Если же $\lambda > r$, количество разрешённых запросов должно быть ограничено сверху значением rt , а избыточные запросы должны отклоняться.

1.2 Алгоритм Token Bucket

Алгоритм Token Bucket является одним из наиболее распространённых механизмов ограничения скорости.

Основная идея заключается в наличии виртуального ведра, содержащего некоторое количество токенов.

Каждый запрос требует один токен.

Токены периодически добавляются в ведро с постоянной скоростью.

Если токен доступен:

- запрос разрешается;
- токен удаляется.

Если токены закончились: запрос отклоняется.

Преимущества:

- хорошо работает с burst-трафиком;
- высокая производительность;
- малое потребление памяти.

Недостатки: допускает кратковременные всплески нагрузки.

Типичные области применения:

- API Gateway;
- балансировщики нагрузки;
- системы аутентификации.

1.3 Алгоритм Leaking Bucket

Leaking Bucket моделирует ведро с отверстием, через которое запросы покидают очередь с постоянной скоростью.

Новые запросы помещаются во внутреннюю очередь.

Система обрабатывает запросы с фиксированной интенсивностью независимо от характера входящего потока.

Преимущества:

- сглаживание нагрузки;
- стабильная скорость обработки.

Недостатки:

- плохо переносит burst-трафик;
- возможны задержки запросов.

Области применения:

- сетевое оборудование;
- маршрутизаторы;
- телекоммуникационные системы.

1.4 Алгоритм Sliding Window Log

Sliding Window Log хранит временные метки всех запросов за заданный интервал.

При поступлении нового запроса:

1. удаляются устаревшие записи;
2. подсчитывается число запросов в окне;
3. принимается решение о допуске.

Преимущества:

- максимальная точность;
- отсутствие эффекта границ окна.

Недостатки:

- высокое потребление памяти;
- сложность обслуживания большого количества клиентов

Области применения:

- биллинговые системы;
- критически важные API;
- платёжные шлюзы.

1.5 Сравнение алгоритмов

Критерий	Token Bucket	Leaking Bucket	Sliding Window Log
Точность	Средняя	Средняя	Высокая
Burst-трафик	Отлично	Плохо	Хорошо
Хранимые данные*	Количество токенов, скорость пополнения, время последнего обновления	Уровень заполнения, скорость утечки, время последнего обновления	Временные метки запросов внутри окна
Производительность	Высокая	Высокая	Средняя
Простота реализации	Высокая	Средняя	Средняя

*В отличие от Token Bucket и Leaking Bucket, объём внутреннего состояния Sliding Window Log зависит от количества запросов, находящихся внутри окна наблюдения. Однако при фиксированном лимите и ограниченном размере окна различие в потреблении памяти может быть незначительным и требует экспериментальной проверки.

Следует отметить, что данное сравнение носит теоретический характер и основано на анализе алгоритмов. Фактические показатели производительности и потребления памяти зависят от конкретной реализации, параметров системы и характера нагрузки. Проверка указанных предположений будет выполнена в экспериментальной части работы.

2) Проектирование системы

2.1 Требования к разрабатываемой системе

Перед началом реализации необходимо определить требования к разрабатываемому программному обеспечению.

Согласно техническому заданию, система должна обеспечивать:

- поддержку нескольких алгоритмов ограничения частоты запросов;
- возможность обслуживания большого количества клиентов одновременно;
- потокобезопасную работу в многопоточной среде;
- поддержку автоматического удаления неактивных клиентов;
- возможность проведения нагрузочного тестирования;
- расширяемость архитектуры для добавления новых алгоритмов ограничения запросов.

Дополнительным требованием являлась возможность унифицированного взаимодействия с любым алгоритмом ограничения запросов. Это необходимо для того, чтобы модульные тесты и нагрузочный тестер могли работать с любой реализацией без изменения собственного кода.

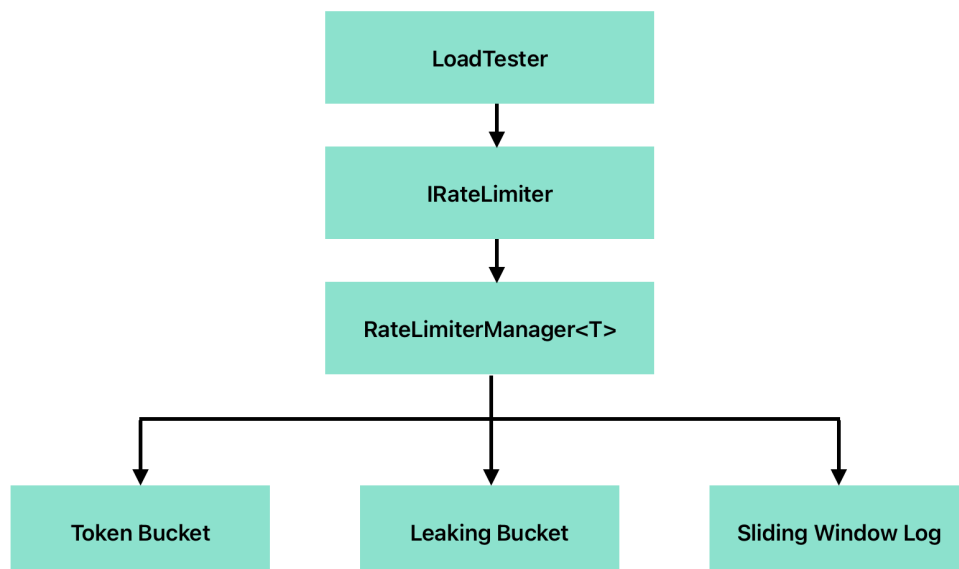
Для выполнения указанных требований была разработана архитектура, основанная на использовании общего интерфейса и шаблонного менеджера клиентов.

2.2 Общая архитектура системы

Разработанная система состоит из трёх основных уровней:

1. Интерфейс ограничения запросов.
2. Реализации алгоритмов ограничения.
3. Менеджер клиентов и подсистема TTL.

Логически архитектура может быть представлена следующим образом:



Ключевой особенностью архитектуры является разделение ответственности между компонентами.

Алгоритмы ограничения запросов отвечают исключительно за логику принятия решения о разрешении либо отклонении запроса.

Управление множеством клиентов, хранение состояний клиентов и механизм TTL вынесены в отдельный компонент `RateLimiterManager`.

Такое разделение существенно упрощает сопровождение и расширение системы.

2.3 Интерфейс `IRateLimiter`

Для унификации взаимодействия с различными алгоритмами был разработан абстрактный интерфейс `IRateLimiter`.

Интерфейс определяет набор операций, которые должна поддерживать любая реализация ограничения запросов. Такой подход исключает реальные задержки в тестах, обеспечивая детерминизм и упрощая модульное тестирование алгоритмов.

Основные методы интерфейса:

```
virtual bool allow(const std::string& key, TimePoint now) = 0;
virtual std::size_t cleanup(TimePoint now);
virtual std::size_t activeKeys() const;
virtual std::size_t evictionCount() const;
virtual std::string name() const = 0;
```

Метод `allow()` выполняет проверку возможности обработки запроса для указанного клиента.

Метод `cleanup()` предназначен для удаления устаревших клиентов по TTL.

Метод `activeKeys()` возвращает количество активных клиентов.

Метод `evictionCount()` используется для сбора статистики по удалённым клиентам.

Метод `name()` позволяет идентифицировать используемый алгоритм во время нагрузочного тестирования.

Особенностью интерфейса является наличие перегрузки `allow()` с явной передачей времени: `allow(key, now)`.

2.4 Шаблонный менеджер клиентов

Центральным элементом архитектуры является класс: `RateLimiterManager<LimiterType>`.

Данный класс реализует интерфейс `IRateLimiter` и обеспечивает поддержку множества клиентов независимо от используемого алгоритма ограничения запросов.

Менеджер хранит ассоциативный контейнер: `std::unordered_map<std::string, KeyState>`, где ключом является идентификатор клиента.

Для каждого клиента хранится структура:

```
struct KeyState { std::unique_ptr<LimiterType> limiter; TimePoint last_access; };
```

- Поле *limiter* содержит экземпляр алгоритма ограничения запросов.
- Поле *last_access* используется механизмом TTL для определения времени последней активности клиента.

Таким образом, каждый клиент получает собственный независимый экземпляр алгоритма ограничения запросов. Подобный подход обеспечивает полную изоляцию состояний различных клиентов.

2.5 Использование шаблонов

Для обеспечения повторного использования кода был выбран шаблонный подход.

Вместо создания отдельных классов (TokenBucketManager, LeakingBucketManager, SlidingWindowManager) используется единый шаблон: RateLimiterManager<LimiterType>.

Например: using TokenBucketLimiter = RateLimiterManager<TokenBucket>;
(или using SlidingLimiter = RateLimiterManager<SlidingWindowLog>;)

Преимущества данного решения:

- отсутствие дублирования кода;
- единая реализация TTL;
- единая реализация хранения клиентов;
- единая реализация статистики;
- упрощение сопровождения проекта.

Фактически алгоритмы отличаются только внутренней логикой принятия решения, тогда как инфраструктурная часть полностью переиспользуется.

2.6 Организация хранения клиентов

Для хранения клиентов используется контейнер: std::unordered_map.

Выбор данного контейнера обусловлен необходимостью быстрого поиска клиента по его идентификатору. Средняя сложность операций:

Операция	Сложность
Поиск	$O(1)$
Добавление	$O(1)$
Удаление	$O(1)$

Использование хеш-таблицы позволяет обслуживать большое количество клиентов без существенного ухудшения производительности. Наиболее практичным является сочетание ленивой и фоновой очистки.

2.7 Реализация TTL

При длительной работе системы возникает проблема накопления неактивных клиентов.

Если клиент перестал отправлять запросы, его состояние остаётся в памяти. При большом количестве клиентов это приводит к постоянному росту потребления памяти.

Для решения данной проблемы используется механизм TTL (Time To Live):

Для каждого клиента хранится время последнего обращения: *last_access*. При каждом вызове *allow(key, now)* данное поле обновляется. Клиент считается неактивным, если выполняется условие: $(now - last_access > ttl)$. После этого его данные могут быть удалены.

2.8 Очистка устаревших клиентов

Удаление неактивных клиентов реализовано методом *cleanup()*.

Во время очистки выполняется обход контейнера клиентов: *while (it != states_.end())*. Для каждого клиента проверяется время его последней активности. Если клиент неактивен дольше установленного TTL, его запись удаляется из контейнера.

Количество удалённых клиентов накапливается в счётчике: *eviction_count_*. Данный счётчик используется при проведении нагрузочных тестов и позволяет анализировать эффективность механизма очистки.

2.9 Потокобезопасность

Одним из требований проекта являлась корректная работа в многопоточной среде.

Для защиты контейнера клиентов используется *std::mutex mutex_*, расположенный внутри *RateLimiterManager*.

Все операции:

- поиска клиента;
- создания клиента;
- обновления *last_access*;
- удаления клиента;

выполняются под защитой данного мьютекса.

Кроме того, каждый алгоритм ограничения запросов содержит собственный *mutex*.

Например, в *Token Bucket* используется: *mutable std::mutex mutex_;*

Аналогичная защита присутствует в *Leaking Bucket* и *Sliding Window Log*.

Таким образом реализована двухуровневая схема синхронизации:

1. Мьютекс менеджера защищает контейнер клиентов.
2. Мьютекс конкретного алгоритма защищает внутреннее состояние клиента.

Такой подход предотвращает возникновение гонок данных и обеспечивает корректную работу системы при одновременном обращении нескольких потоков.

2.10 Выводы по проектированию

В результате проектирования была разработана модульная архитектура, позволяющая сравнивать различные алгоритмы ограничения частоты запросов в идентичных условиях.

Ключевыми особенностями разработанной системы являются использование общего интерфейса `IRateLimiter`, шаблонного менеджера клиентов `RateLimiterManager`, механизма TTL и встроенной поддержки потокобезопасности.

Выбранная архитектура обеспечивает расширяемость системы и позволяет добавлять новые алгоритмы ограничения запросов без изменения существующей инфраструктуры проекта.

В следующей главе рассматриваются особенности реализации алгоритмов Token Bucket, Leaking Bucket и Sliding Window Log.

3) Реализация системы

3.1 Общие сведения о реализации

Разработанная система реализована на языке программирования C++23 и состоит из набора независимых компонентов, взаимодействующих через общий интерфейс.

Исходный код проекта разделён на несколько логических модулей:

- `include/rate_limiter` — интерфейсы и объявления классов;
- `src` — реализации алгоритмов и инфраструктурных компонентов;
- `tests` — модульные тесты;
- `benchmarks` — нагрузочные тесты;
- `load_tester` — приложение для генерации различных профилей нагрузки.

Подобная структура обеспечивает удобство сопровождения проекта и позволяет независимо развивать отдельные компоненты системы.

3.2 Интерфейс ограничения запросов

Для унификации взаимодействия с различными алгоритмами был разработан абстрактный интерфейс `IRateLimiter`.

Интерфейс определяет единый набор операций для всех реализаций и позволяет использовать различные алгоритмы ограничения запросов взаимозаменяемым образом.

Основными функциями интерфейса являются:

- проверка возможности выполнения запроса;
- удаление устаревших клиентов;
- получение статистической информации;
- идентификация используемого алгоритма.

Наличие общего интерфейса позволило реализовать нагрузочный тестер и модульные тесты независимо от конкретного алгоритма ограничения запросов.

3.3 Менеджер клиентов

Для поддержки множества пользователей был реализован шаблонный класс `RateLimiterManager`. Основной задачей данного компонента является управление жизненным циклом клиентов и хранение экземпляров алгоритмов ограничения запросов.

Для каждого клиента создаётся собственный экземпляр алгоритма, что обеспечивает независимость состояний различных пользователей.

Менеджер также отвечает за:

- регистрацию новых клиентов;
- обновление времени последней активности;
- удаление устаревших записей по TTL;
- сбор статистики по количеству удалённых клиентов.

Использование шаблонного класса позволило избежать дублирования кода для различных алгоритмов ограничения запросов.

3.4 Реализация алгоритма Token Bucket

Алгоритм Token Bucket реализован в виде отдельного класса, содержащего информацию о количестве доступных токенов, максимальной ёмкости ведра и скорости пополнения.

При поступлении запроса выполняется вычисление количества токенов, накопленных с момента последнего обращения. После обновления состояния выполняется проверка наличия свободного токена.

Если токен доступен, запрос разрешается и количество токенов уменьшается на единицу. В противном случае запрос отклоняется.

Данный алгоритм обеспечивает поддержку кратковременных всплесков нагрузки и характеризуется низким потреблением памяти.

3.5 Реализация алгоритма Leaking Bucket

Алгоритм Leaking Bucket моделирует ведро с постоянной скоростью утечки.

Для каждого клиента хранится текущий уровень заполнения ведра. Перед обработкой нового запроса вычисляется количество запросов, покинувших систему с момента предыдущего обращения.

Если после обновления состояния ведро не превышает допустимую ёмкость, запрос принимается. В противном случае выполняется отклонение запроса.

Основным преимуществом данного подхода является сглаживание входящего потока и поддержание практически постоянной скорости обработки запросов.

3.6 Реализация алгоритма Sliding Window Log

В реализации Sliding Window Log для каждого клиента хранится журнал временных меток запросов, принадлежащих текущему окну наблюдения.

При поступлении нового запроса выполняется удаление устаревших записей, время которых выходит за пределы окна. После очистки определяется количество запросов, находящихся внутри текущего временного интервала.

Если число запросов меньше установленного лимита, временная метка нового запроса добавляется в журнал и запрос разрешается. В противном случае запрос отклоняется.

Основным преимуществом данного алгоритма является высокая точность соблюдения ограничений. В отличие от алгоритмов, использующих усреднение или накопление токенов, Sliding Window Log принимает решение на основе фактической истории запросов внутри окна наблюдения.

Объём памяти, используемый алгоритмом, определяется количеством временных меток, находящихся в текущем окне. При фиксированном размере окна и ограниченном лимите число хранимых записей остаётся ограниченным, что делает данный подход пригодным для практического применения.

3.7 Реализация механизма TTL

Для предотвращения неограниченного роста потребления памяти в системе реализован механизм TTL (Time To Live).

Для каждого клиента сохраняется момент его последней активности. Если клиент не выполнял запросов в течение времени, превышающего установленное значение TTL, его данные считаются устаревшими и могут быть удалены.

Удаление устаревших клиентов выполняется методом `cleanup()`, реализованным в менеджере клиентов.

Использование TTL позволяет ограничить объём занимаемой памяти даже при большом количестве уникальных клиентов.

3.8 Обеспечение потокобезопасности

Поскольку система ориентирована на работу в многопоточной среде, особое внимание было уделено вопросам синхронизации доступа к данным.

Для защиты общих структур данных используются мьютексы стандартной библиотеки C++. Потокобезопасность обеспечивается на двух уровнях:

1. Защита контейнера клиентов в `RateLimiterManager`.
2. Защита внутреннего состояния конкретного алгоритма ограничения запросов.

Такой подход предотвращает возникновение гонок данных и обеспечивает корректную работу системы при одновременном обращении нескольких потоков к одному или нескольким клиентам.

3.9 Выводы по реализации

Реализована модульная система rate-limiting с несколькими алгоритмами, TTL и поддержкой большого числа клиентов. Единый интерфейс и шаблонный менеджер исключают дублирование кода и унифицируют управление клиентами. Архитектура готова для экспериментального сравнения алгоритмов под разной нагрузкой.

4) Тестирование системы

4.1 Цели тестирования

После завершения реализации системы было проведено модульное тестирование основных компонентов проекта. Целью тестирования являлась проверка корректности работы реализованных алгоритмов ограничения запросов, механизма TTL и поддержки многопоточного доступа. Основными задачами тестирования являлись:

- проверка соблюдения установленных лимитов запросов;
- проверка восстановления состояния алгоритмов во времени;
- проверка обработки всплесков нагрузки;
- проверка корректности удаления неактивных клиентов;
- проверка потокобезопасности реализации.

Для автоматизации тестирования использовалась библиотека Google Test.

4.2 Проверка корректности ограничения запросов

Одним из основных требований к системе является соблюдение установленных ограничений частоты запросов. Для каждого алгоритма выполнялась серия тестов, проверяющих корректность обработки запросов при различных значениях лимитов и временных параметров. В ходе тестирования проверялись следующие сценарии:

- выполнение запросов в пределах установленного лимита;
- превышение допустимого количества запросов;
- восстановление возможности выполнения запросов после истечения необходимого временного интервала.

Проведённые проверки подтвердили корректность реализации базовой логики ограничения запросов.

4.3 Проверка обработки всплесков нагрузки

Для алгоритмов, поддерживающих накопление разрешений на выполнение запросов, была проведена проверка обработки кратковременных всплесков нагрузки.

В ходе тестирования моделировалась ситуация, при которой клиент длительное время не выполняет запросов, после чего отправляет серию запросов за короткий промежуток времени.

Проверка показала соответствие поведения алгоритмов ожидаемым теоретическим свойствам и корректную обработку burst-нагрузки.

4.4 Проверка механизма TTL

Для предотвращения неограниченного роста потребления памяти в системе реализован механизм удаления неактивных клиентов. В процессе тестирования проверялись следующие сценарии:

- создание нового клиента;
- обновление времени последней активности;
- удаление клиента после превышения времени жизни;
- повторное создание клиента после удаления.

Полученные результаты подтвердили корректность работы механизма TTL и процедуры очистки устаревших записей.

4.5 Проверка многопоточного доступа

Поскольку система ориентирована на работу в многопоточной среде, особое внимание было уделено проверке потокобезопасности.

В ходе тестирования выполнялось одновременное обращение нескольких потоков к одному и тому же клиенту, а также к различным клиентам. Проверялись следующие свойства:

- отсутствие аварийного завершения программы;
- отсутствие взаимных блокировок;
- отсутствие повреждения внутренних структур данных;
- соблюдение ограничений запросов при конкурентном доступе.

Результаты тестирования подтвердили корректность использования механизмов синхронизации и устойчивость системы к многопоточному доступу.

4.6 Выводы по тестированию

Разработанный набор модульных тестов позволил проверить корректность реализации всех основных компонентов системы.

В ходе тестирования была подтверждена работоспособность алгоритмов ограничения запросов, механизма TTL и подсистемы многопоточного доступа.

Полученные результаты позволяют перейти к следующему этапу исследования — нагрузочному тестированию и сравнительному анализу алгоритмов в различных сценариях эксплуатации.

5) Методика нагрузочного тестирования

5.1 Цели экспериментального исследования

После подтверждения корректности работы системы было проведено нагрузочное тестирование реализованных алгоритмов ограничения запросов.

Основной целью экспериментов являлось сравнение алгоритмов Token Bucket, Leaking Bucket и Sliding Window Log в условиях различных профилей нагрузки.

В ходе исследования предполагалось оценить влияние используемого алгоритма на производительность системы, точность ограничения запросов и эффективность использования ресурсов.

5.2 Используемые метрики

Для количественной оценки эффективности алгоритмов использовался набор метрик, характеризующих различные аспекты работы системы.

Throughput

Показатель Throughput характеризует количество запросов, обработанных системой за единицу времени. Метрика измеряется в запросах в секунду (requests per second, req/s) и позволяет оценить общую производительность реализации.

Высокое значение Throughput свидетельствует о способности алгоритма эффективно обрабатывать интенсивный поток запросов.

Latency

Latency характеризует время выполнения отдельного запроса. В рамках исследования анализировались: среднее время обработки запроса, медианное время обработки (p50), 99-й перцентиль (p99).

Использование p99 позволяет оценить поведение системы в наиболее неблагоприятных условиях и выявить редкие задержки, которые могут быть незаметны при анализе среднего значения.

Active Keys

Метрика Active Keys показывает количество клиентов, состояние которых хранится в системе в текущий момент времени. Данная характеристика позволяет оценить влияние различных сценариев нагрузки на объём поддерживаемого состояния.

Eviction Count

Метрика Eviction Count показывает количество клиентов, удалённых механизмом TTL. Данный показатель позволяет оценить эффективность механизма очистки и интенсивность обновления клиентских данных при различных профилях нагрузки.

5.3 Генерация тестовой нагрузки

Для проведения экспериментов использовалось отдельное приложение нагрузочного тестирования, входящее в состав проекта.

Нагрузочный тестер обеспечивает генерацию различных сценариев пользовательской активности и позволяет проводить идентичные испытания для всех реализованных алгоритмов. В рамках работы использовались следующие профили нагрузки:

- Uniform;
- Burst;
- Diurnal;
- Sparse + Burst;
- Zombie.

Каждый сценарий моделирует определённый тип поведения клиентов, встречающийся в реальных информационных системах.

5.4 Равномерная нагрузка (Uniform)

Сценарий Uniform моделирует ситуацию, при которой запросы поступают с постоянной интенсивностью на протяжении всего эксперимента.

Данный профиль соответствует работе сервисов с относительно стабильным количеством пользователей.

Равномерная нагрузка позволяет оценить базовую производительность алгоритмов и выявить накладные расходы, возникающие при стандартном режиме эксплуатации.

5.5 Всплесковая нагрузка (Burst)

Сценарий Burst моделирует резкое увеличение интенсивности запросов за короткий промежуток времени. Подобное поведение характерно для:

- массовых обращений пользователей после публикации новости;
- запуска рекламной кампании;
- кратковременных пиков активности.

Данный сценарий позволяет оценить способность алгоритмов корректно обрабатывать всплески нагрузки и соблюдать установленные ограничения.

5.6 Суточный профиль нагрузки (Diurnal)

Сценарий Diurnal моделирует изменение активности пользователей в течение суток.

Интенсивность запросов постепенно увеличивается в периоды максимальной активности и уменьшается в периоды низкой нагрузки.

Подобный профиль характерен для большинства пользовательских сервисов и позволяет оценить работу алгоритмов при плавно изменяющейся нагрузке.

5.7 Разреженная нагрузка со всплесками (Sparse + Burst)

В данном сценарии длительные периоды низкой активности чередуются с кратковременными всплесками запросов.

Подобный режим характерен для телеметрических систем, IoT-устройств и некоторых типов фоновых сервисов.

Основной целью испытания является проверка способности алгоритмов корректно восстанавливать своё состояние после длительных периодов простоя.

5.8 Нагрузка типа Zombie

Сценарий Zombie моделирует большое количество клиентов, которые выполняют ограниченное число запросов и затем длительное время остаются неактивными.

Данный профиль нагрузки используется для оценки эффективности механизма TTL и способности системы освобождать память, занимаемую устаревшими состояниями клиентов.

Особое внимание в данном эксперименте уделяется метрикам Active Keys и Eviction Count.

5.9 Методика проведения экспериментов

Для обеспечения объективности результатов каждый эксперимент проводился при одинаковых настройках системы и одинаковых параметрах нагрузки.

Каждый сценарий запускался отдельно для всех трёх алгоритмов ограничения запросов.

Полученные результаты использовались для построения сравнительных таблиц и последующего анализа эффективности алгоритмов.

5.10 Выводы по методике тестирования

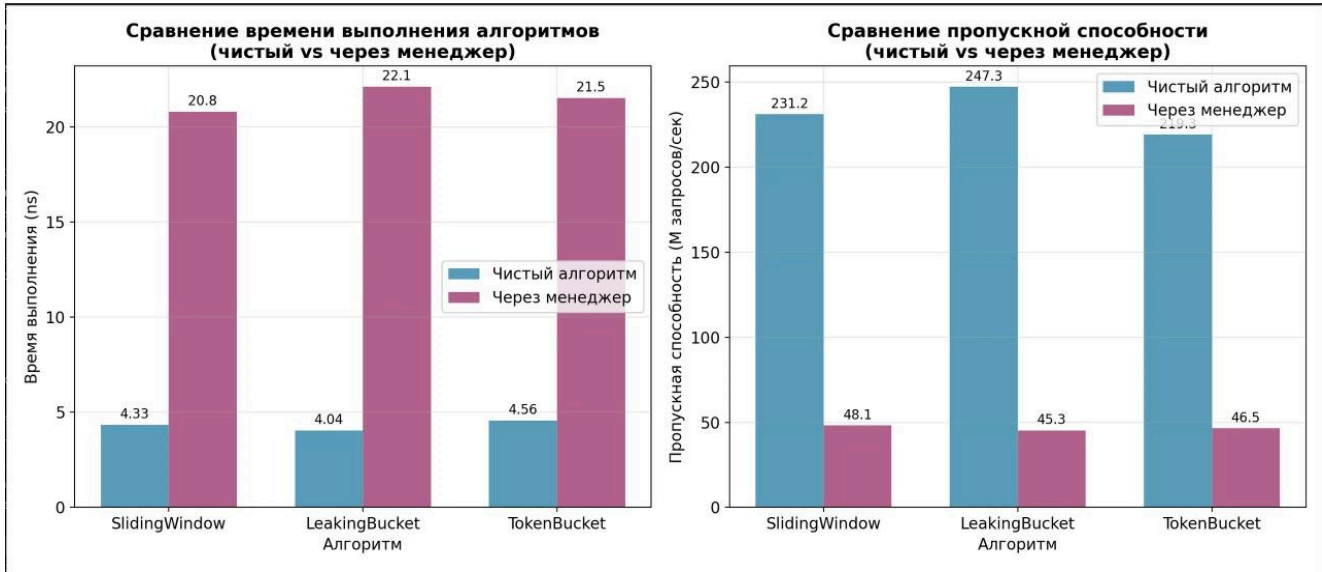
Разработанная методика позволяет оценить поведение алгоритмов ограничения запросов в условиях, приближённых к реальным сценариям эксплуатации.

Использование нескольких профилей нагрузки и набора количественных метрик обеспечивает возможность объективного сравнения реализованных алгоритмов и выявления их сильных и слабых сторон.

6) Результаты экспериментального исследования

6.1 Сравнение алгоритмов без учёта менеджера клиентов

На первом этапе была выполнена оценка производительности непосредственно алгоритмов ограничения запросов без использования менеджера клиентов. Полученные результаты представлены на рисунке 6.1:



Среднее время обработки одного запроса составило:

Алгоритм	Время выполнения, нс
Sliding Window Log	4.33
Leaking Bucket	4.04
Token Bucket	4.56

Полученные значения отличаются незначительно. Наименьшее время выполнения продемонстрировал алгоритм Leaking Bucket, однако различия между всеми тремя реализациями находятся в пределах единиц наносекунд.

Пропускная способность алгоритмов без учёта инфраструктурных компонентов составила:

Алгоритм	Время выполнения, нс
Sliding Window Log	231.2
Leaking Bucket	247.3
Token Bucket	219.3

Наибольшую пропускную способность показал алгоритм Leaking Bucket.

Таким образом, при рассмотрении исключительно внутренней логики алгоритмов существенных различий в производительности обнаружено не было.

6.2 Влияние менеджера клиентов

На втором этапе выполнялось измерение производительности полной системы с использованием класса RateLimiterManager. Среднее время обработки запроса составило:

Алгоритм	Время выполнения, нс
Sliding Window Log	20.8
Leaking Bucket	22.1
Token Bucket	21.5

Соответствующая пропускная способность снизилась до:

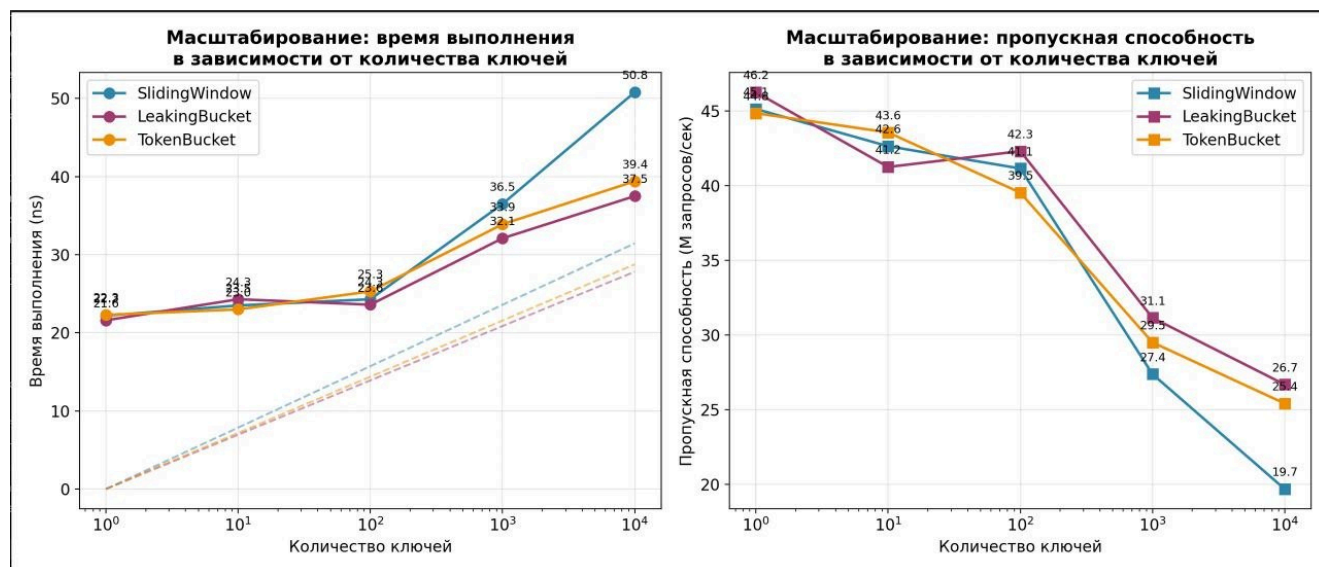
Алгоритм	Время выполнения, нс
Sliding Window Log	48.1
Leaking Bucket	45.3
Token Bucket	46.5

Во всех случаях наблюдается значительное снижение производительности по сравнению с измерениями для отдельных алгоритмов.

Данный результат свидетельствует о том, что основная часть времени выполнения приходится не на вычисления внутри алгоритмов ограничения запросов, а на работу инфраструктурных компонентов системы: поиск клиентов, обновление времени активности, синхронизацию доступа к контейнерам и управление состоянием клиентов.

6.3 Масштабирование по количеству клиентов

Для оценки поведения системы при увеличении числа клиентов было проведено исследование зависимости времени выполнения от количества ключей.



Эксперименты проводились для следующих значений: 1, 10, 100, 1000, 10000 клиентов.

Полученные результаты показывают устойчивый рост времени обработки запроса с увеличением числа клиентов. Наиболее заметный рост продемонстрировал Sliding Window Log:

Количество ключей	Время выполнения, нс
1	22.3
10000	50.8

Для Token Bucket аналогичный показатель составил 39.4 нс, а для Leaking Bucket — 37.5 нс.

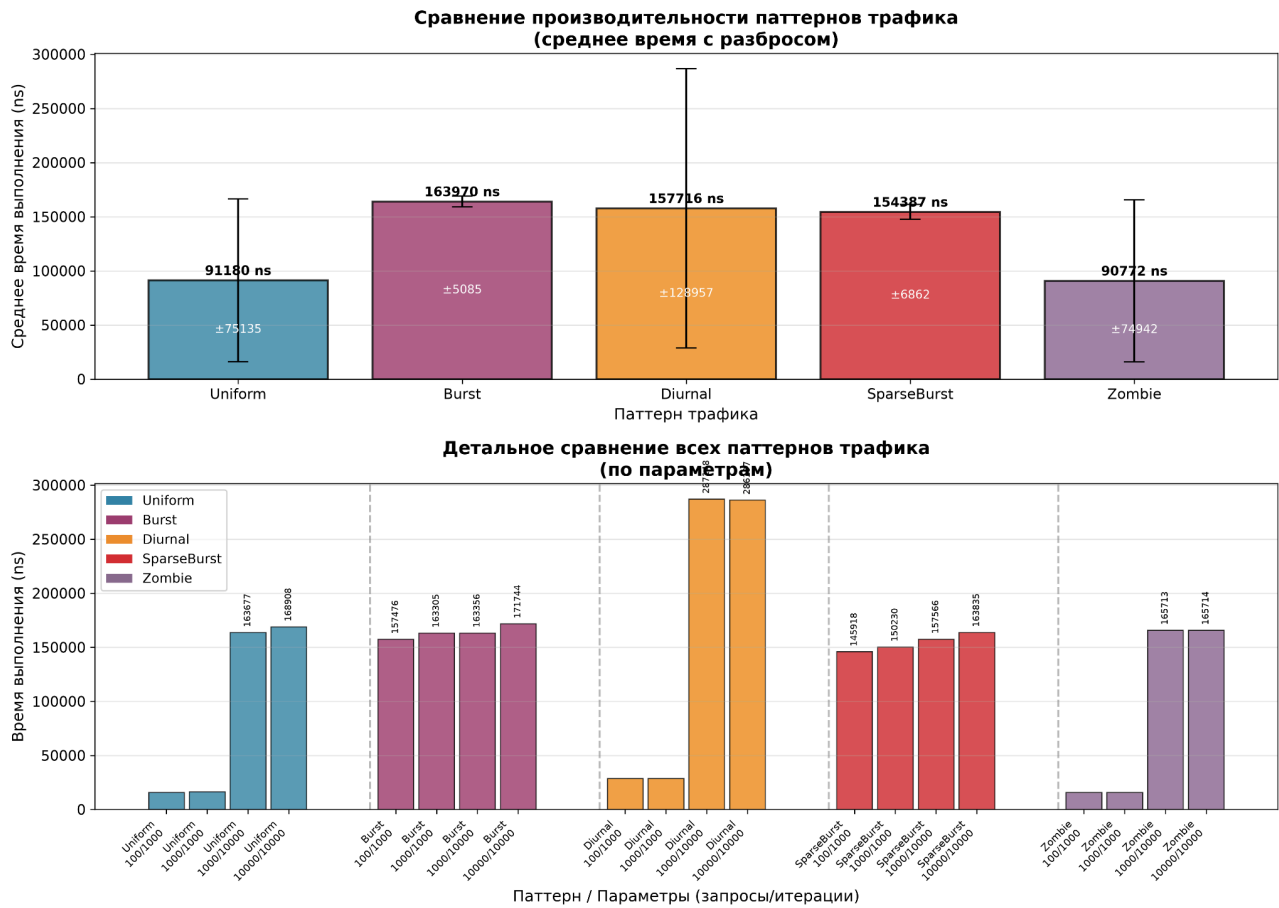
Соответственно наблюдается снижение пропускной способности системы. При 10000 клиентах пропускная способность составила:

Алгоритм	Запросов/сек
Sliding Window Log	19.7
Leaking Bucket	26.7
Token Bucket	25.4

Несмотря на снижение производительности, все алгоритмы сохраняют работоспособность при росте количества клиентов на четыре порядка.

6.4 Анализ различных профилей нагрузки

Для оценки поведения системы в условиях, приближённых к реальной эксплуатации, были исследованы пять профилей нагрузки: Uniform, Burst, Diurnal, Sparse Burst, Zombie.



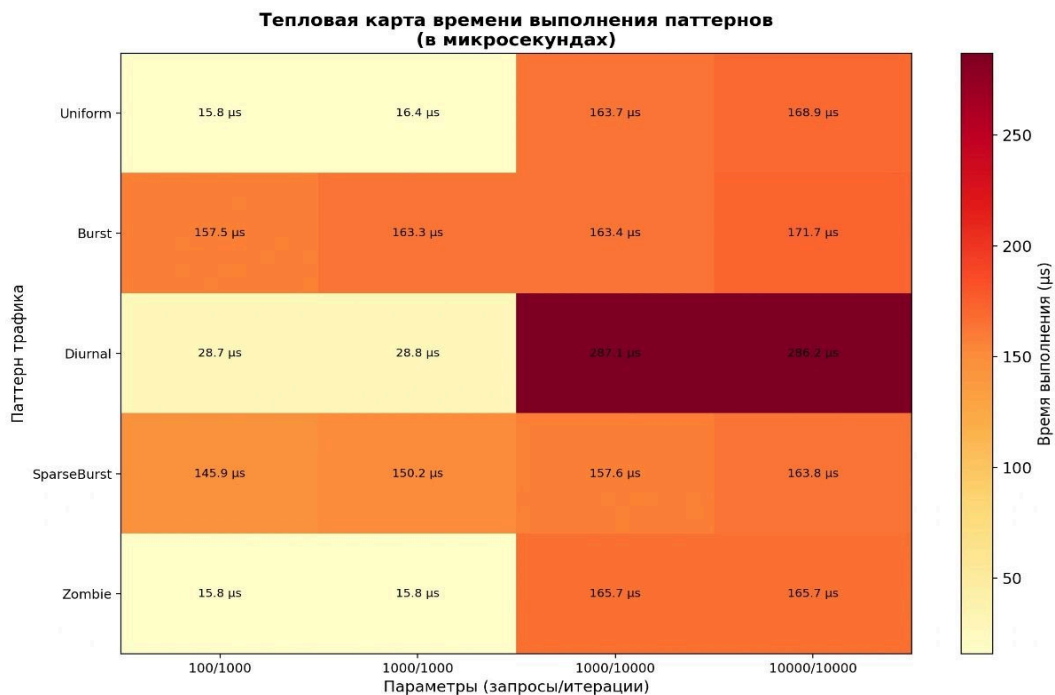
Среднее время обработки запросов составило:

Паттерн	Uniform	Burst	Durnal	Sparse Burst	Zombie
Среднее время, мкс	91.2	164.0	157.7	154.4	90.7

Наиболее низкие задержки наблюдаются для профилей Uniform и Zombie. Наибольшие значения времени выполнения зафиксированы для Burst, Diurnal и Sparse Burst.

6.5 Тепловой анализ результатов

Для более детального исследования была построена тепловая карта времени выполнения.



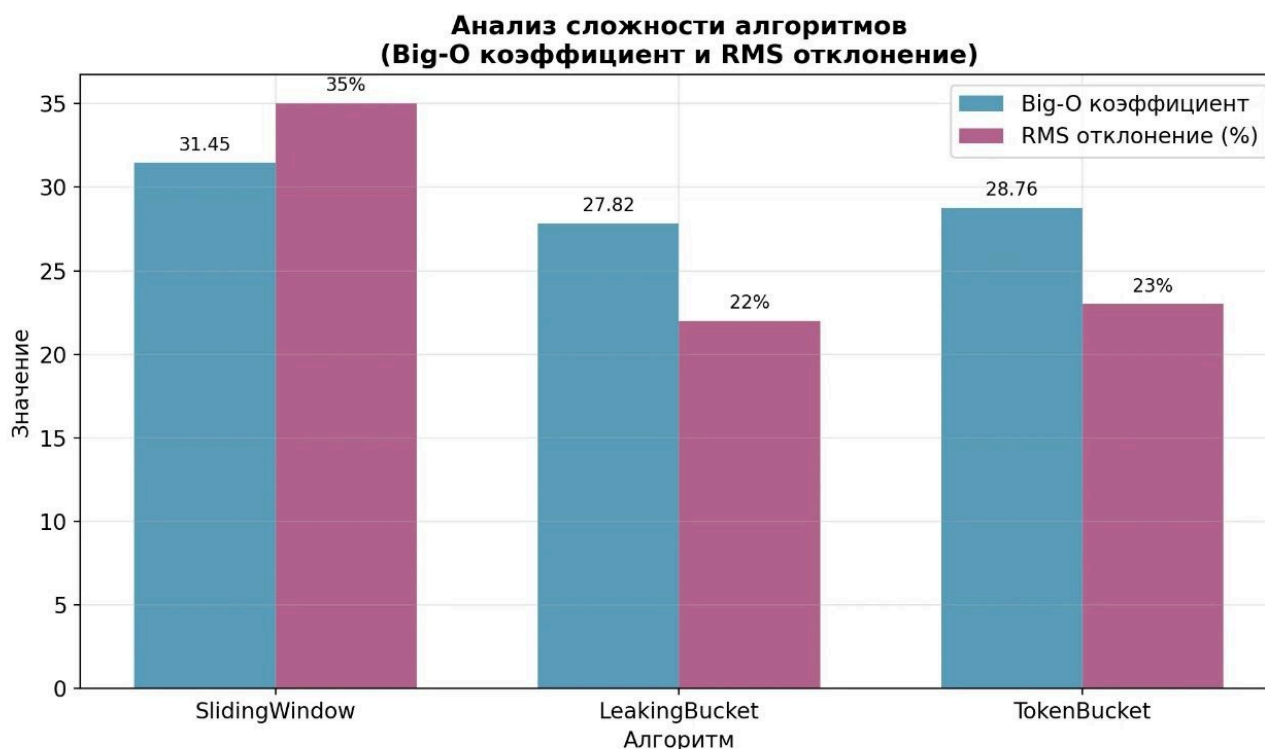
Наиболее благоприятными конфигурациями являются сценарии Uniform и Zombie при низкой интенсивности запросов, где время обработки составляет около 16 мкс. Наиболее затратным сценарием оказался профиль Diurnal при высокой нагрузке:

Конфигурация	Время, мкс
Diurnal 1000/10000	287.1
Diurnal 10000/10000	286.2

Полученные результаты свидетельствуют о том, что плавное изменение интенсивности нагрузки оказывает более заметное влияние на систему, чем равномерный поток запросов.

7) Анализ результатов

7.1 Сравнение реализованных алгоритмов

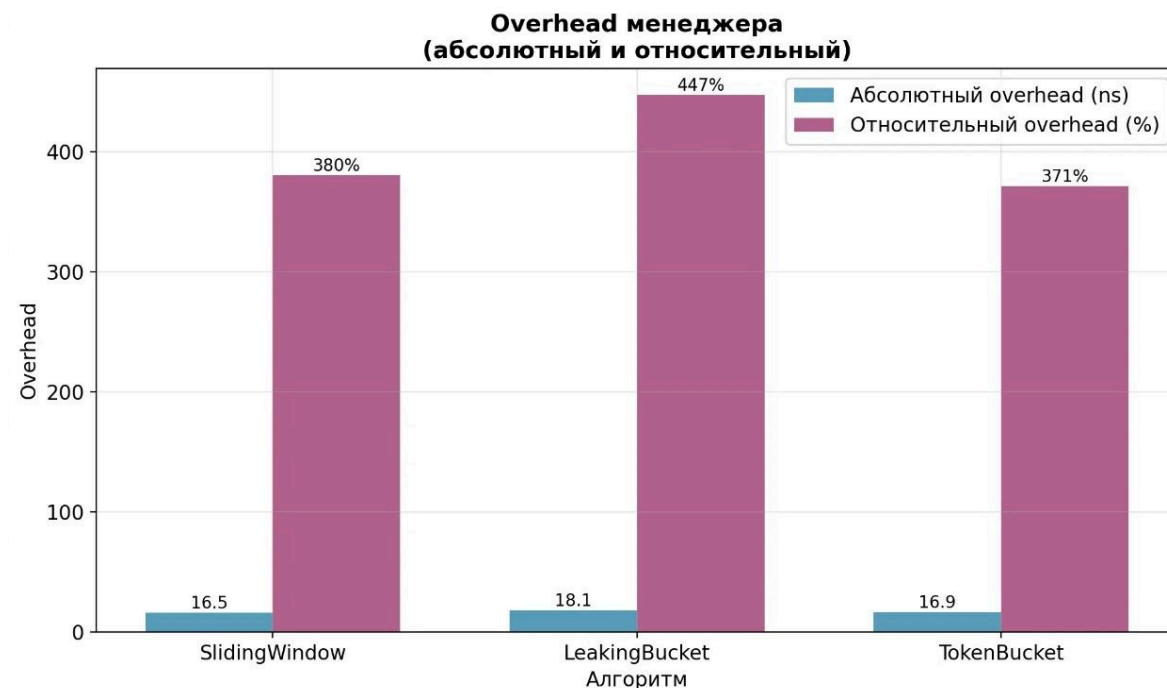


Проведённые эксперименты показали, что различия между алгоритмами Token Bucket, Leaking Bucket и Sliding Window Log значительно меньше, чем предполагалось на этапе теоретического анализа.

Во всех испытаниях результаты трёх алгоритмов находятся в близких диапазонах как по времени выполнения, так и по пропускной способности. Это объясняется тем, что вычислительная сложность внутренней логики алгоритмов невелика, а объём выполняемых операций для одного запроса остаётся небольшим. Следовательно, при выбранных параметрах эксперимента производительность определяется не столько используемым алгоритмом ограничения запросов, сколько общей архитектурой системы.

7.2 Влияние инфраструктурных компонентов

Наиболее важным результатом исследования стало выявление существенного влияния менеджера клиентов на итоговую производительность системы.



Добавление слоя RateLimiterManager приводит к увеличению времени обработки примерно в четыре-пять раз. Для всех алгоритмов наблюдается сходная величина накладных расходов:

Алгоритм	Overhead
Sliding Window Log	380 %
Leaking Bucket	447 %
Token Bucket	371 %

Данный результат показывает, что основным фактором, ограничивающим производительность системы, является не алгоритм ограничения запросов, а операции управления состоянием клиентов.

7.3 Масштабируемость системы

При увеличении количества клиентов производительность системы постепенно снижается.

Тем не менее рост времени выполнения происходит достаточно плавно. Даже при увеличении количества клиентов с 1 до 10000 система сохраняет приемлемые показатели производительности. Это свидетельствует о корректной работе выбранной архитектуры и эффективности использования хеш-таблицы для хранения клиентских состояний.

7.4 Анализ профилей нагрузки

Наиболее простыми для системы оказались сценарии Uniform и Zombie. В данных режимах нагрузка распределяется относительно равномерно либо значительная часть клиентов быстро становится неактивной.

Наиболее сложным сценарием оказался Diurnal. Причиной является постоянное изменение интенсивности запросов, приводящее к более частым обновлениям внутреннего состояния клиентов и увеличению количества операций управления состоянием системы.

Сценарии Burst и Sparse Burst занимают промежуточное положение между указанными крайними случаями.

7.5 Практические рекомендации

На основании проведённых экспериментов можно сделать следующие выводы.

Если приоритетом является максимальная пропускная способность, предпочтительным выбором является Leaking Bucket.

Если требуется наиболее простая и хорошо изученная реализация, целесообразно использовать Token Bucket.

Если критически важно максимально точное соблюдение ограничений, предпочтительным является Sliding Window Log.

Однако в рамках данной реализации различия между алгоритмами оказываются существенно меньше влияния инфраструктурных компонентов системы.

Следовательно, дальнейшая оптимизация должна быть направлена прежде всего на снижение накладных расходов менеджера клиентов и механизмов синхронизации, а не на замену одного алгоритма ограничения запросов другим.

7.6 Выводы по анализу результатов

Проведённое исследование показало, что разработанная система успешно обеспечивает ограничение частоты запросов для различных профилей нагрузки и большого количества клиентов.

Основным результатом работы является установление того факта, что в рамках рассматриваемой архитектуры производительность системы определяется преимущественно инфраструктурными компонентами, тогда как различия между отдельными алгоритмами ограничения запросов оказываются сравнительно небольшими.

Полученные результаты подтверждают работоспособность предложенной архитектуры и позволяют сформулировать рекомендации по её дальнейшему развитию и оптимизации.

Заключение

В ходе выполнения работы была разработана потокобезопасная система ограничения частоты запросов, поддерживающая алгоритмы Token Bucket, Leaking Bucket и Sliding Window Log. Для управления состоянием клиентов реализован универсальный класс RateLimiterManager, обеспечивающий хранение клиентских данных, поддержку механизма TTL и удаление неактивных записей.

Для проверки корректности работы системы был разработан набор модульных тестов, охватывающий основные сценарии использования, включая проверку ограничений запросов, обработку всплесков нагрузки, работу механизма TTL и многопоточный доступ.

В рамках экспериментального исследования было выполнено сравнение реализованных алгоритмов при различных профилях нагрузки. Полученные результаты показали, что различия между алгоритмами по времени выполнения и пропускной способности оказываются незначительными по сравнению с накладными расходами инфраструктурных компонентов системы. Основное влияние на производительность оказывает работа менеджера клиентов и механизмов синхронизации.

Проведённое исследование также показало, что разработанная архитектура сохраняет работоспособность при увеличении количества клиентов и корректно функционирует в различных сценариях нагрузки. Наиболее ресурсоёмкими оказались профили Burst, Diurnal и Sparse Burst, тогда как Uniform и Zombie продемонстрировали наименьшие задержки обработки запросов.

Таким образом, поставленные в работе цели были достигнуты: реализованы и исследованы различные алгоритмы ограничения частоты запросов, разработан нагрузочный тестер, проведены эксперименты и выполнен анализ полученных результатов. Полученные данные могут быть использованы при выборе алгоритма ограничения запросов и дальнейшем развитии системы.